

ECE 49595 SD I – Verification & Validation

Spring 2026

Objectives

Objective 1: Verify that only valid university users can activate accounts through .edu email verification.

Linked Requirements: FR-003, SYS-103

Measurement Criteria:

- verification emails are sent only to valid .edu addresses
- each verification link contains a unique token

Objective 2: Verify that the recommendation system generates relevant ranked teammate recommendations based on selected game, preferred role, and availability.

Linked Requirements: FR-001, SYS-101

Measurement Criteria:

- all returned recommendations match the selected game
- at least 80% of the top 5 recommended users match the requester's preferred role and availability criteria
- the recommendation list is displayed in ranked order

Objective 3: Verify that linked gaming account statistics are retrieved and displayed accurately on user profiles.

Linked Requirements: FR-004, SYS-104

Measurement Criteria:

- a linked test account successfully retrieves known API data
- the displayed profile rank matches the expected value returned by the game API

Objective 4: Verify that users can successfully browse, create, and interact with core campus community features, including forum discussions and lobby cards.

Linked Requirements: FR-006, FR-008

Measurement Criteria:

- users can create, view, and reply to forum posts successfully
- interaction paths for forum and lobby features complete without broken navigation or missing content
- lobby cards display required information, including game title, host username, rank range, player slots, and join action

Objective 5: Verify that authorized users can exchange real-time messages and retrieve previous chat history successfully.

Linked Requirements: FR-005, SYS-105

Measurement Criteria:

- authorized user pairs can send and receive direct messages
- each sent message is delivered within 3 seconds

Objective 6: Verify that the system delivers actionable notifications and routes users to the correct related page.

Linked Requirements: FR-007

Measurement Criteria:

- the correct notification is generated for a new message, recommendation, or group update
- selecting the notification opens the correct related page

Prioritization

Product Capability	Priority (High, Medium, Low)	Brief Justification
University Email Verification (FR-003)	High	Core feature. It keeps CampusGG limited to real students. If it fails, users may not trust the platform.
Recommendation System (FR-001)	High	Core feature. It helps users find teammates. If the matches fail, the main goal of the project also fails.
Gaming Statistics Synchronization (FR-004)	High	This feature strengthens trust in teammate selection by showing verified game rank and profile data. It also depends on outside APIs.
Forum Page (FR-008)	High	Main user feature. Users need it to post, reply, and interact with others on campus.
Lobby Feed (FR-006)	High	Important part of the matchmaking workflow. Users rely on accurate display of lobby details to decide whether to join. Incorrect rendering would directly reduce usability and trust.
User Interface (FR-002)	Medium	Important for usability. Users need it to move across the system, but it depends on the core features working correctly first.
Mobile Application (FR-009)	Low	Useful for convenience, but it is not the main focus of the current project scope. The web platform and core matching features are more important.

Full RVTM

📌 Team 6 - RVTM Workshop

Test Approach

Unit Testing:

Backend: Testing the matchmaking algorithm's logic, testing hashing for `.edu` email token generation, and verifying data models.

Frontend: Testing individual UI components: registration forms, recommendation results, lobby cards, forum post inputs, chat inputs, notifications, website protection, and profile stat display.

Integration Testing:

Validating the communication between our backend server and external third-party gaming APIs. Testing database queries to ensure user profiles, chat messages, and forum posts are successfully written to and retrieved from the database.

System Testing:

Account creation → `.edu` verification → linking a gaming account → finding a teammate → sending a direct message.

Manual vs. Automated Testing:

Manual Testing: This includes UI layout, navigation, forum and lobby interaction, notification, profile display, and qualitative UI/UX evaluation.

Automated Testing: Backend and frontend unit tests, API checks, routing checks, recommendation checks, verification checks, and repeated regression tests.

Non-functional Testing:

Latency: Real-time messaging delivery and notification routing will be measured to ensure sub-5-second latency.

Security: `.edu` verification, login protection

Usability: Main flows are clear, no broken navigation, accessible UI

Reliability: Forum / Chat / Lobby data save and reload correctly

Regression Testing:

Running repeat tests during development after updates and bug fixes.

Main focus on: `.edu` email verification, recommendation system, gaming statistics synchronization, forum page, lobby feed, messaging, and notifications.

Validation Plan

User acceptance testing approaching: We can have a small group of verified students (10 students) at Purdue. These users will test realistic scenarios including creating an account with a university email, setting gaming preferences, viewing recommended users, joining a lobby, sending messages, and interacting with notifications. UAT will occur at the end of each major development iteration so feedback can be incorporated into the next cycle. We will aim to meet as a group with users (if possible) to note their feedback. Test participants will be asked to complete defined tasks and then provide feedback on whether the system behaved as expected and whether the features felt useful and intuitive. Any repeated issues or unmet expectations found during UAT will be logged, prioritized, and addressed before the next validation round.

UAT Success Metrics

- At least 85% of test users successfully complete core tasks without developer assistance
- At least 80% of participants report that the system meets their expectations for its main features
- Fewer than 10% of submitted tasks result in critical user-blocking errors
- At least 75% of users indicate they would feel comfortable using the platform again

Usability and performance evaluations: Just like before, we can have another or the same group of 10 verified students. These students would not test the more visual and accessible aspects of CampusGG. For usability, participants will be observed while completing common workflows such as navigating the home page, finding a recommended user, creating or joining a lobby, and opening a message thread. The team will note areas of confusion, extra clicks, hesitation, and navigation errors. For performance, the team will test how quickly major features respond, including page loads, recommendation generation, messaging delivery, and notification display. These evaluations will be repeated after significant UI changes or backend improvements so that both user experience and technical responsiveness continue to improve over time. Just as before, we can review this with the group for each round of validation and ensure the previous logs of bugs or issues have been addressed in CampusGG.

Usability and Performance Success Metrics

- At least 85% task completion rate on major workflows
- Average time to complete basic tasks remains within acceptable targets
- Messaging is delivered within 3 seconds under normal test conditions
- Forum, lobby, and profile pages load within 2 to 3 seconds under expected user load
- User-reported confusion points decrease across testing iterations
- Error rates in common actions remain below 5%

Tools, Environments & Test Data Sets

Testing Tools:

- Jest: unit testing
- React Testing Library: frontend component testing
- Playwright: end-to-end UI testing
- Postman: API testing

Environments:

- We will use a local development environment for early feature checks and unit testing.
- We will also use an integration environment to test how the frontend, backend, database, messaging service, and outside APIs work together. This will let us test realistic user flows before release without affecting the main deployed version.

Test Data Sets:

- We will use realistic and representative test data for the main features. This includes `.edu` test accounts for user verification, user profiles with different games, roles, and availability for recommendation testing, and known game stat data from test API accounts for profile testing.
- We will also use sample forum posts, lobby records, chat history, and notifications to test community and communication features. The test data will reflect normal CampusGG use, but it will be anonymized and will not include real private user information.

Version Control and Test Management Tools:

- We will use GitHub for version control and GitHub Actions to run repeated tests during development and support regression testing. GitHub will help the team manage code changes and track updates. GitHub Actions will help automate testing and keep test runs consistent across development.

Tool Configuration:

- Jest and React Testing Library will run in the local development environment for unit and component testing, and they can also be triggered in GitHub Actions.
- Playwright will run end-to-end UI tests in the staging or integration environment.
- Postman collections will be organized by feature area, such as authentication, recommendation, and messaging, for repeated API testing.

Test Result Archiving:

- Test results will be archived in GitHub Actions logs, local test reports, and Postman collection results.
- UI test results and feedback from validation sessions will also be stored for later review.
- Failed tests and bugs will also be tracked in the team's issue tracker for follow-up and retesting.